Connecting and Automating Analysis & Planning

From backlog to release — one chat, every step.

Feedback!
Share with us:
1. Learnings you've had so far
2. Whitespaces you might still find

Areas Impacting Engineering Velocity & Quality

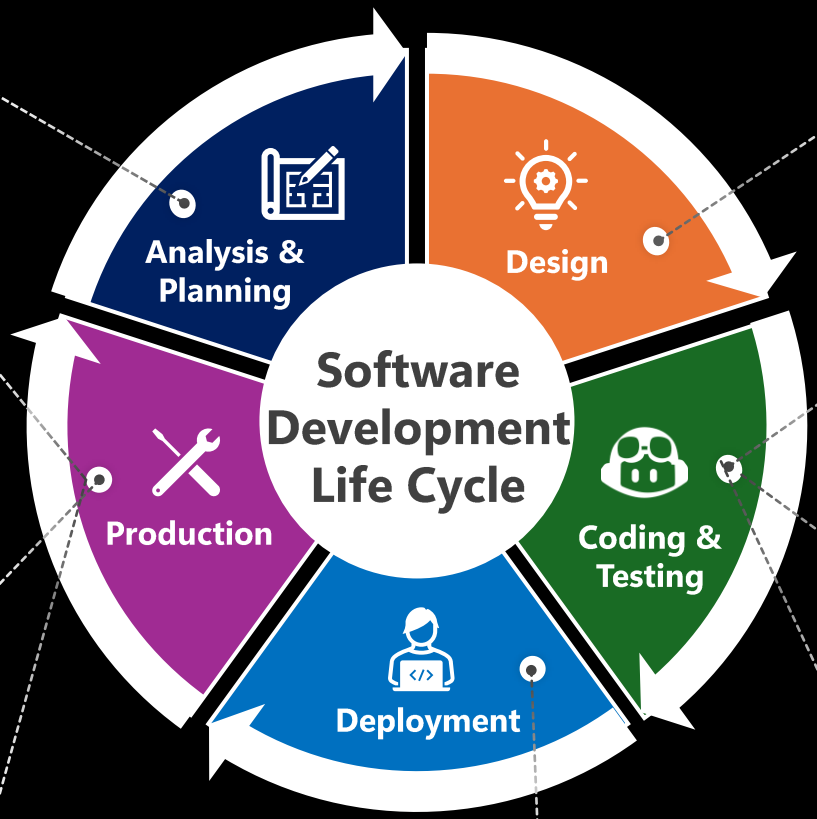
Present State: Pain Points in Our SDLC

Manual sprint communication & ADO hygiene issues

Engineering hours wasted on manual reports and info searches

Gaps in Live Site Incident monitoring

Fragmented SFI management reduces efficiency



Inefficient or inconsistent Design Review processes

Challenge in Measuring Coding Productivity

Manual, fragmented Accessibility Testing and fixes

Different coverage of Testing across teams

Risk of Deploying outdated features, impacting customer experience

Accelerate Productivity by embedding AI

Our Proposal: AI-Driven Solutions for SDLC Improvement

Planning Agent & Dashboard

Summarize sprint plans, create and update ADO items, and ensure hygiene

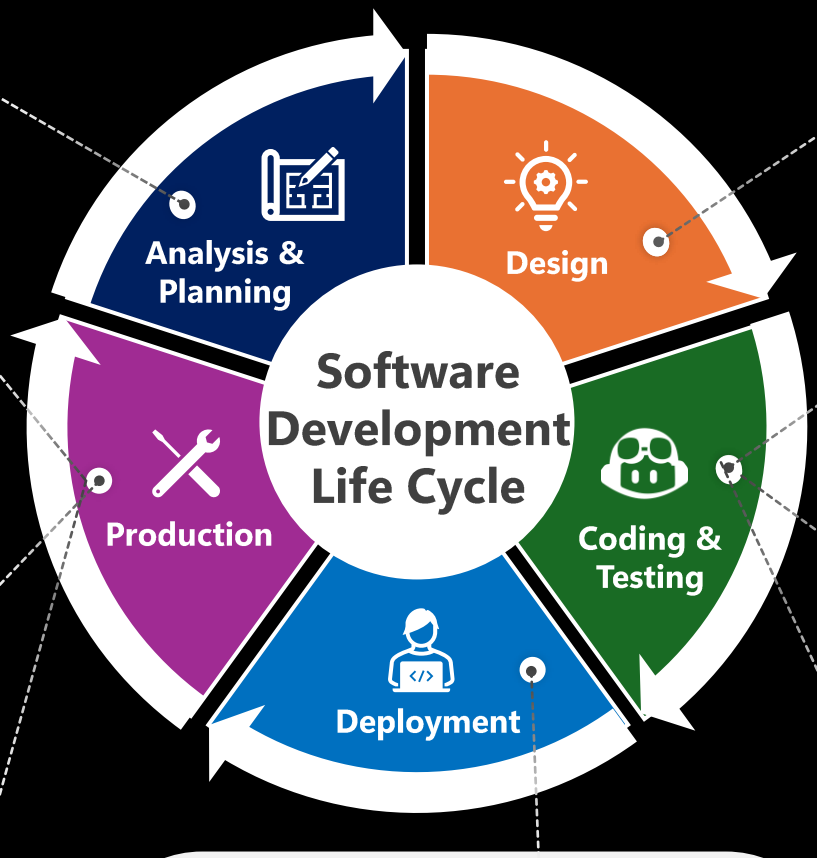
Onboarding Agent, & DRI Agent

SRE Agent

Self-serve live site incident to provide 24×7 monitoring with proactive alerts, and close alerting gaps with recommendations

s360 Agent

Leverage an Agent built by C+E SFI Squad to manage SFI with two specialized agents integrated into the SWE agent



Release Freshness Agent

Track production freshness and automatically follow-ups on delayed deployments and pending changes

Design Review Agent

Provide early feedback on design, architecture, and security using historical data to help refining designs before peer review

Productivity Agent

Automate measure the coding productivity

Accessibility Agent

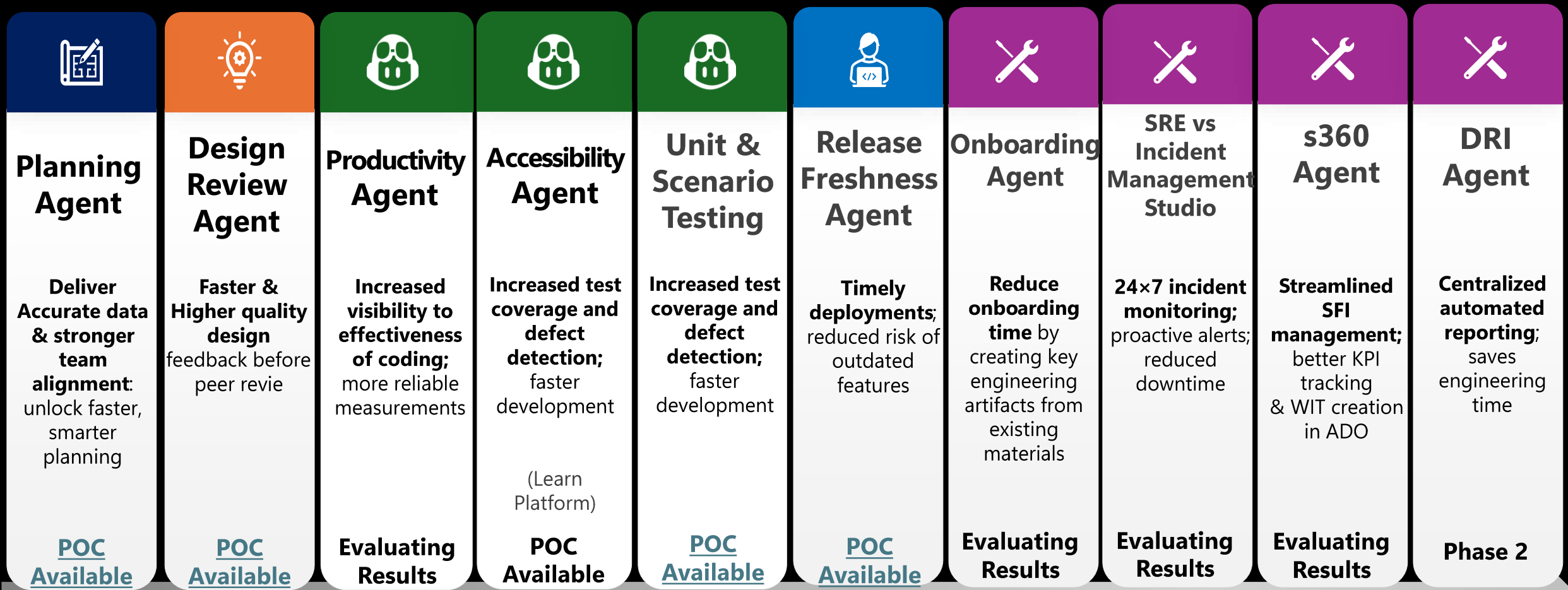
Automate accessibility checks and fixes for faster, more reliable results

AX&E AI Testing

Automated AI Unit & Scenario testing

Improve Software Development Life Cycle (SDLC)

Accelerate AX&E Engineering productivity by embedding AI into everyday work



* ***Zero Production Touch**: Keep work in Learn, maintain only, no further expansion*

Why wire agents into your trackers

Context is scattered across tools

Code lives in GitHub, work lives in Jira or Azure DevOps, decisions live in chat. Agents that only see code make decisions blind to the plan and ship the wrong thing fast.

Manual handoffs break the loop

Developers copy-paste between IDE, tracker, and PR. Each switch is friction — and a chance for the audit trail to drift from what really shipped.

Releases without traceable evidence

When stories close without linked artifacts, compliance reviews fall back to screenshots and trust. Agents can produce evidence — pytest output, run JSON, sign-off comments — but only if you wire them into the tracker.

The AI-powered developer platform

Scale and governance

Plan	Code	Verify	Deploy	Operate
Issues	Editor/IDEs	Pull Requests	Actions (CI/CD)	Metrics/ROI
Projects	Copilot CLI	/code-review	Releases	Spark Runtime
Spaces	/coding	/secure + /auto-fix	Apps	SRE Agent
/planning	Mission control	/testing	AI Workflows	MCP Integrations
Teams + Slack	Custom agents	Code quality	Custom agents	
Boards			Pipelines	

Agent control plane

MCP + Custom Agents for the backlog

Set the context once. Drive every ticket.
Define your agents once in .github/agents/, and let Copilot orchestrate the entire delivery lifecycle. By wiring the **Story-Implementer** and **Sprint Sign-off** agents to the **Jira MCP** and **Azure DevOps MCP**, Copilot can drive every backlog item through a consistent loop: **read** → **reason** → **comment** → **transition**.
Today, we'll use these two MCPs to interact directly with Jira and Azure DevOps from the IDE—reading work items, updating status, adding comments, and signing off stories. Once connected, Copilot becomes the execution layer that ensures every ticket follows the same automated flow.

What is the Story-Implementer agent

- Takes a backlog ticket → drives it through read, reason, comment, transition
- Defined as a Markdown contract under .github/agents/ with scoped MCP tools
- Runs against the patched local Jira/ADO MCP servers — every call is logged and auditable
- Human-approved by design: can’t close a sprint; every transition shows up in Jira/ADO

Lab 06

Backlog Workflow: Jira MCP, Azure DevOps MCP & Custom Agents

Today's Lab

Setup

Connect to the Jira or Azure DevOps MCPs

Part 1 — Backlog Demo

Part 1 — Read a real Jira ticket through the patched MCP server and pick one to drive.

Part 2 — Connect Agents to the MCP Server

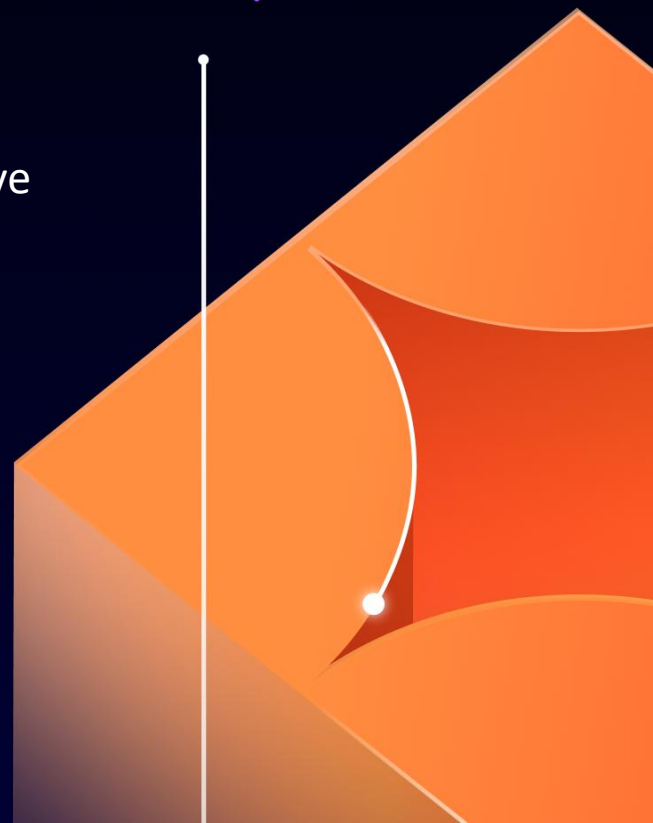
Part 2 — Connect the Jira and Azure DevOps MCP servers and verify agents can see your live backlog.

Part 3 — Custom Agents

Run the Story-Implementer agent through a full read → reason → comment → transition cycle.

Part 4 — Sprint Sign-off End-to-End

Run the Sprint Sign-off agent across the backlog and review the transitions and comments it logs.



Thank you